



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

LABORATORY RECORD NOTEBOOK

Name :ISHANA SABRISH.....

Register No :231501063.....

Year / Semester : IV / 7

Batch : 2023 - 2027

Branch : Artificial Intelligence and Data Science

Degree : B. Tech.

Subject Code : AD23731

Subject Name : Foundations of Agentic AI

Academic Year : 2026 – 2027 (ODD)



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the mini-project work done by.....ISHANA SABRISH Register No.....231501063.. of VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in charge

Submitted for the Practical Examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

1. Introduction
2. Problem Statement and Business Case
3. User and Stakeholder Analysis
4. Proposed Agentic AI Solution
5. Dataset / Knowledge Sources
6. Tools and Technologies Used
7. Agent Architecture
8. Agent Workflow and Orchestration
9. Agentic RAG Implementation
10. System Implementation
11. Experimental Evaluation and Results
12. Comparison, Innovation and Scalability
13. Conclusion and Future Work

1. INTRODUCTION

1.1 Background and Motivation

Software delivery has traditionally been a linear, human-mediated process in which requirements, design, implementation, testing and deployment are carried out by different people using different tools, with knowledge passed on informally through documents, meetings and tribal memory. Generative AI has recently shown that large language models (LLMs) can draft requirements, write code, generate tests and even suggest deployment steps individually. However, a single chatbot invoked repeatedly for each of these tasks has no persistent context, no defined handoff between roles, and no accountability trail linking a shipped feature back to the requirement that produced it. Agentic AI - where multiple specialised, tool-using agents collaborate under an orchestrator - offers a way to mirror the Software Development Lifecycle (SDLC) itself, with each agent responsible for one phase and a shared memory layer preserving context across the whole run.

AgentFlow is built to explore this idea directly: a plain-English feature request is carried through six specialised agents that together reproduce the Requirements, Design, Coding, Testing, Review and DevOps phases of the SDLC, while three human approval gates ensure that no code reaches production without a person's explicit sign-off.

1.2 Need for Agentic AI

- Single-prompt LLM usage loses context at every re-invocation; an agentic pipeline keeps a shared memory of past decisions, requirements and code across steps.
- Real SDLC work needs specialisation - a requirements analyst does not reason the same way as a security reviewer or a release engineer. Dedicated agents with narrow, well-defined responsibilities produce more reliable, checkable output than one generalist model.
- Traceability and accountability are structurally difficult when a single actor (human or AI) performs every step. An agent crew with logged inputs/outputs at each stage makes the full requirement-to-release trail auditable.
- Human oversight does not disappear with automation; it must be relocated to the highest-leverage decision points. Approval gates formalise where a human must intervene, rather than leaving oversight to chance.

1.3 Project Objectives

1. Design and implement a six-agent, orchestrator-driven pipeline that carries a feature request through Requirement, Design, Coding, Testing, Review and DevOps stages.
2. Implement three mandatory human approval gates (Product Owner, Developer, SRE) that pause the pipeline for Approve / Request changes / Reject decisions.

3. Provide a shared memory layer so every agent can read prior decisions and project context, and so requirement-to-deployment traceability can be reconstructed after the fact.
4. Build a role-based Stakeholder View so six different real-world roles (Client/Product Owner, Project Manager, Developer, QA Engineer, DevOps/SRE, End User) each see only the information and controls relevant to them.
5. Enforce guardrails - a step budget per agent, mandatory human sign-off before release, no secrets exposed to agents, and a full audit log - and surface them visibly in the UI.
6. Evaluate the system's reliability, traceability and usefulness against a defined set of functional and qualitative metrics.

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 Problem Definition

Feature delivery in most software teams suffers from three linked problems: (a) requirements ambiguity introduced during translation from stakeholder language into engineering language; (b) loss of design rationale as work passes from designer to developer to tester, so decisions cannot later be explained; and (c) manual, spreadsheet-based traceability that makes it slow and error-prone to answer 'which requirement produced this code, and was it tested and reviewed before release?'. These problems compound as the number of concurrent feature requests grows, and they are the direct cause of scope creep, late-discovered defects, and releases that must be rolled back because no one agent (human or otherwise) held the full picture.

2.2 Business / Real-World Context

Organisations adopting AI-assisted development today typically bolt a code-generation assistant onto an otherwise unchanged process: a developer asks a copilot to write a function, but the requirement it came from, the design decision behind it, and the tests that validate it are still tracked (if at all) in separate, disconnected systems. AgentFlow's business case is that a project-scoped, multi-agent SDLC pipeline with built-in traceability and approval gates removes this disconnect: it gives Product Owners, Developers, QA and SRE staff one system of record for how a request moved from idea to release, while keeping the decision rights (approve, request changes, reject) firmly with humans at the three points that matter most - accepting the requirement, approving the code, and releasing to production.

3. USER AND STAKEHOLDER ANALYSIS

3.1 Target Users

AgentFlow is aimed at small-to-medium engineering teams and academic/prototype settings where a single feature request needs to be taken from a plain-English description to a reviewed, deployable plan, with clear points at which a human must approve the work before it proceeds.

3.2 Stakeholder Identification

Table 1: Stakeholder Identification

Stakeholder	Needs	Touchpoint in AgentFlow
Client / Product Owner	Wants features delivered as described, with no surprises.	Reviews the Requirement Agent's user stories at Gate 1 before any design or code work begins.
Project Manager	Needs visible status and realistic estimates.	Reads the orchestrator's live activity log and pipeline status; sees blockers as soon as a step fails.
Developer	Wants help, not replacement; owns the final code.	Reviews the Coding Agent's implementation and the Review Agent's audit at Gate 2 before code is accepted.
QA Engineer	Needs coverage of the cases that actually break the system.	Curates and extends the Testing Agent's generated test suites and edge-case list.
DevOps / SRE	Cares about safe, reversible releases.	Gates the DevOps Agent's deployment plan at Gate 3 and owns the rollback decision.
End User	Cares only that the software works and is safe.	Protected indirectly by the Review Agent's quality/security audit and the three human gates.

3.3 User Requirements

- A single plain-English entry point for submitting a feature request.
- Visibility into which agent is working, which has finished, and which is waiting on a human decision.
- The ability to approve, request changes (with a comment that is fed back to the agent), or reject at each of the three gates.
- A role-specific view so each stakeholder is not overwhelmed by information meant for a different role.
- A durable, queryable record of every requirement, design decision, code artefact, test and deployment step, and the links between them.

4. PROPOSED AGENTIC AI SOLUTION

4.1 Overall Solution

AgentFlow is a web application in which a user's plain-English feature request is processed by an orchestrator that invokes six specialised agents in sequence - Requirement, Design, Coding, Testing, Review and DevOps - each implemented as a Supabase Edge Function that calls the Lovable AI Gateway (an LLM inference endpoint) with a role-specific prompt. Each agent receives the outputs of every prior agent plus relevant entries from a shared memory store, and returns structured JSON that the frontend renders into agent-specific views. Three human approval gates - after Requirements, after Review, and before DevOps - pause the pipeline until a Product Owner, Developer or SRE respectively records an Approve, Request changes or Reject decision.

4.2 Key Features

- Dashboard for submitting new requests and tracking the status of past runs.
- Run View: a horizontal six-agent pipeline with live status per agent (waiting, running, done, needs approval, failed) and a side panel for full agent output.
- Stakeholder View: a role switcher (six roles) that filters the same run data into a role-appropriate layout and exposes only that role's approval controls.
- Traceability page linking every requirement to its design decision, code file, test and deployment step.
- Shared Memory page exposing the decisions and context agents draw on, so humans can audit what the agents 'know'.
- A visible Safety/Guardrails panel: per-agent step budget, mandatory human sign-off before release, a no-secrets-to-agents policy, and a full audit log.

4.3 Agent-Based Approach

Rather than a single model asked to 'build the feature', the problem is decomposed along the natural boundaries of the SDLC. Each agent is given a narrow responsibility, a fixed input contract (its own inputs plus the accumulated shared context) and a fixed output contract (structured JSON matching a schema the UI already knows how to render). This mirrors how human teams specialise, makes each agent's output independently checkable, and allows a failed or rejected step to be re-run in isolation - with a review comment - without re-running the entire pipeline.

5. DATASET / KNOWLEDGE SOURCES

AgentFlow does not train on a static labelled dataset; its 'knowledge source' is a combination of (a) the live, structured project context generated and accumulated during each run, and (b) the general-purpose reasoning and code-generation capability of the underlying LLM served through the Lovable AI Gateway.

5.1 Structured Project Context (Shared Memory)

- The original feature request text submitted by the user for the current run.
- Every prior agent's structured output within the same run (user stories, design decisions, code, tests, review findings, deployment plan) - i.e., intra-run context.
- Persisted memory_items from previous runs on the same project - architectural conventions, prior decisions and constraints that should not be re-litigated on every request - i.e., cross-run, project-level context.
- Human review comments captured at each approval gate, which are fed back into the same agent on a re-run.

5.2 Evaluation Reference Material

For evaluation purposes (Section 11), a small internal set of 8-10 representative feature requests of varying complexity was authored by the project team (e.g., 'add a password-reset flow', 'add CSV export to the reports page') and used consistently across pipeline runs so that agent output quality, gate friction and traceability could be compared on a like-for-like basis.

6. TOOLS AND TECHNOLOGIES USED

Table 2: Tools and Technologies Used

Layer	Technology	Role
Application platform	Lovable	AI-assisted full-stack build and deployment environment for the React frontend and Supabase backend.
Frontend	React, TypeScript, Tailwind CSS, shadcn/ui	Dashboard, Run View, Stakeholder View, Traceability and Shared Memory pages; component-driven, responsive UI.
Backend / database	Supabase (Postgres, Edge Functions, Row Level Security, Realtime)	Stores projects, runs, agent_steps, approvals, memory_items and activity_log; hosts each agent as a serverless Edge Function; pushes live status updates to the Run View.
AI inference	Lovable AI Gateway (LLM access)	Each of the six agents calls the gateway with a role-specific system prompt and the accumulated shared context, returning structured JSON.
Orchestration logic	A dedicated orchestrator Edge Function	Decides the next agent to run, assembles shared context, retries a failed step once, enforces the per-agent step budget, and escalates to a human on repeated failure.
Version control	Git / GitHub (via Lovable's GitHub sync)	Source history and collaboration on the generated codebase.

7. AGENT ARCHITECTURE

7.1 Agent Components

- Requirement Agent (RA) - converts the raw feature request into user stories with acceptance criteria.
- Design Agent (DA) - proposes system architecture, a data model and an API contract for the accepted stories.
- Coding Agent (CA) - writes implementation code that satisfies the design's API contract and the story's acceptance criteria.
- Testing Agent (TA) - writes unit tests against the generated code and enumerates edge cases the tests may not cover.
- Review Agent (RV) - audits the code for quality issues, security concerns and coverage against the original requirement.
- DevOps Agent (DO) - produces a deployment plan, CI configuration and rollback steps for the reviewed, approved code.

7.2 Tools Available to Agents

Each agent is a stateless Supabase Edge Function invocation: its only 'tool' is the Lovable AI Gateway call itself, constrained by a strict input schema (its own role's fields plus shared context) and a strict output schema (the structured JSON the frontend expects). No agent is given direct database write access, filesystem access or network access beyond the LLM call; all persistence is performed by the orchestrator after validating an agent's returned JSON, which keeps the blast radius of any single agent's output small and auditable.

7.3 Memory

Shared memory has two horizons. Within a run, the orchestrator assembles a running context object containing every previous agent's output plus any human review comments, and passes it whole to the next agent - this is how the Design Agent 'knows' the accepted user stories, and how the Coding Agent 'knows' the approved design. Across runs, selected decisions are written to the `memory_items` table (e.g., 'the project uses Postgres with Row Level Security for multi-tenant isolation') and are surfaced to every agent on later runs of the same project, avoiding re-derivation or contradiction of established conventions.

7.4 Reasoning Flow

Each agent call follows the same shape: (1) receive role-specific instructions plus the accumulated shared context; (2) reason over that context to produce the artefact its role is responsible for (stories, design, code, tests, review findings, or a deployment plan); (3) return a structured JSON object matching a fixed schema, including any flags the agent could not resolve with confidence; (4) the orchestrator validates the JSON, persists it to

agent_steps, appends an entry to activity_log, and updates the agent's status on the Run View.

7.5 Agent Interactions

Agents never call one another directly; all interaction is mediated by the orchestrator, which enforces the fixed hand-off order Requirement -> Design -> Coding -> Testing -> Review -> DevOps, inserting a human approval gate at three of those hand-offs. This keeps the interaction graph simple, makes every hand-off inspectable in the activity log, and means a 'Request changes' decision at a gate can re-invoke exactly the agent responsible, with the human's comment appended to its input, without disturbing the rest of the run.

8. AGENT WORKFLOW AND ORCHESTRATION

8.1 Workflow Design

A run begins when a user submits a feature request on the Dashboard, creating a row in runs with status 'pending'. The orchestrator then drives the run through a fixed state machine: Requirement -> [Gate 1] -> Design -> Coding -> Testing -> Review -> [Gate 2] -> [Gate 3] -> DevOps -> Complete. Each agent transition writes a row to agent_steps (agent, input, output, status, tokens, duration) and an entry to activity_log, giving a complete, replayable history of the run.

8.2 Task Decomposition

The feature request is decomposed along SDLC phase boundaries rather than along feature sub-tasks: each agent performs one phase completely (for the current story) before handing off, rather than several agents each doing a slice of every phase. This keeps each agent's prompt focused and its output independently reviewable by the human role most qualified to judge it.

8.3 Agent Coordination

Coordination is centralised in the orchestrator rather than negotiated between agents. The orchestrator is responsible for: selecting the next agent; assembling that agent's input from shared context; enforcing a step budget per agent (a maximum number of LLM calls/retries for that phase); retrying a failed step exactly once automatically; and escalating to a human (rather than retrying again) if the same step fails twice.

8.4 Orchestration Implementation

The orchestrator is implemented as its own Supabase Edge Function, invoked after each agent completes (or after a human records a gate decision). It reads the run's current state and the latest agent_steps rows, determines the next action per the state machine above, and either invokes the next agent function, re-invokes the current agent with an appended human comment (on 'Request changes'), or halts the run (on 'Reject' or on a second consecutive failure) with a clear reason recorded in activity_log.

8.5 Approval Gates

Table 3: Approval Gates

Gate	Position	Approving Role	On Approve	On Request Changes	On Reject
Gate 1	After Requirement Agent	Product Owner	Proceed to Design Agent	Re-run Requirement Agent with comment	Halt run
Gate 2	After Review Agent	Developer	Proceed to Gate 3	Re-run Coding Agent (and Review) with comment	Halt run
Gate 3	Before DevOps Agent	SRE	Proceed to DevOps Agent	Return to Review Agent with comment	Halt run

9. AGENTIC RAG IMPLEMENTATION

9.1 Retrieval Architecture

AgentFlow implements a lightweight, structured form of retrieval-augmented generation rather than a vector-similarity search over unstructured documents. The 'corpus' retrieved from is the project's own structured history: `memory_items` and prior `agent_steps` rows for the same project, filtered by relevance (matching project id, and where applicable matching story or component keywords) before being assembled into the context object passed to the next agent. This keeps retrieval precise and auditable, which matters more here than broad recall, since the 'documents' being retrieved are prior engineering decisions that must not be contradicted.

9.2 Knowledge Retrieval

Before each agent call, the orchestrator queries `memory_items` for the current project (ordered by recency and, where tagged, by topical match to the current story) and concatenates the retrieved items with the full in-run history from `agent_steps`. This two-tier retrieval - intra-run (always included) and cross-run (selectively retrieved) - is what allows, for example, the Design Agent on a new request to remain consistent with a data-model decision made in an earlier, unrelated run on the same project.

9.3 Generation Process

The retrieved context, together with the requesting agent's role-specific system prompt and output schema, is sent as a single call to the Lovable AI Gateway. The agent's generation is constrained to the structured JSON schema for its role, which both simplifies downstream rendering and makes hallucinated or malformed fields easy to detect and reject before they are persisted.

9.4 Retrieval-Quality Evaluation

Retrieval quality was assessed qualitatively during development by checking, on a set of multi-run test projects, whether a later run's Design Agent output remained consistent with an earlier run's recorded architectural decisions (e.g., choice of authentication scheme), and whether irrelevant `memory_items` (from unrelated stories) leaked into an agent's context and degraded its output. Precision of retrieval (useful items retrieved / items retrieved) was prioritised over recall, since the shared-memory store is small and project-scoped rather than an open-ended external corpus.

10. SYSTEM IMPLEMENTATION

10.1 Functional Prototype Screenshots

The screenshots below show the working first slice: the Run View pipeline with all six agents (RA, DA, CA, TA, RV, DO) shown complete for a sample request, the activity log and the human approval gates panel (top), and the Requirement Agent's structured output open in the side panel - the generated user stories with their IDs, roles, wants, reasons and acceptance criteria (bottom).

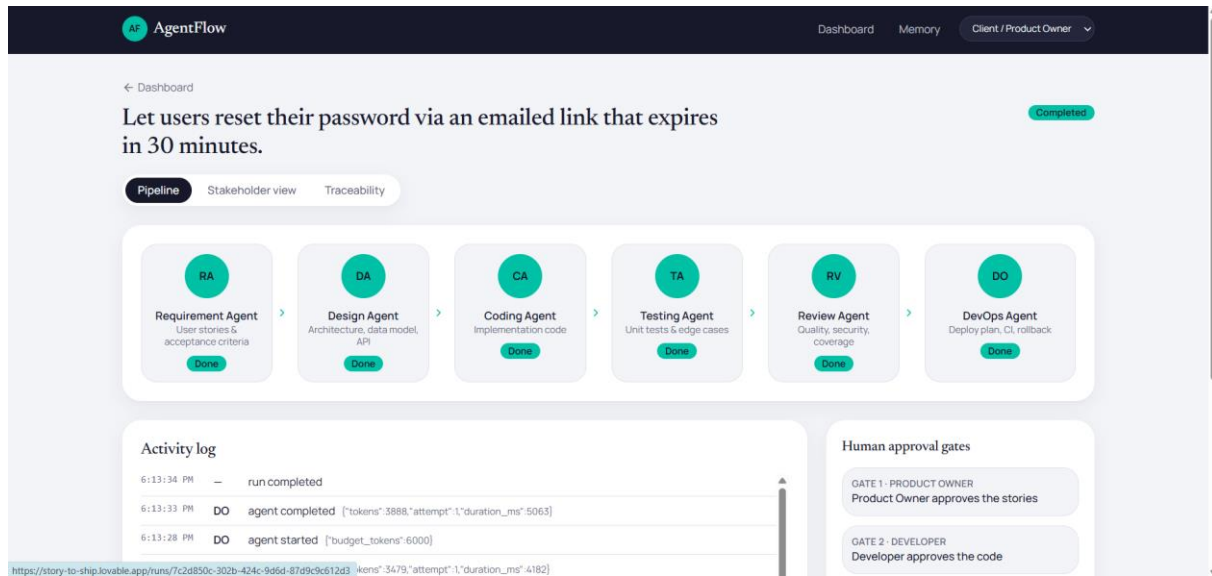


Figure 1: AgentFlow Run View - pipeline status, activity log and approval gates

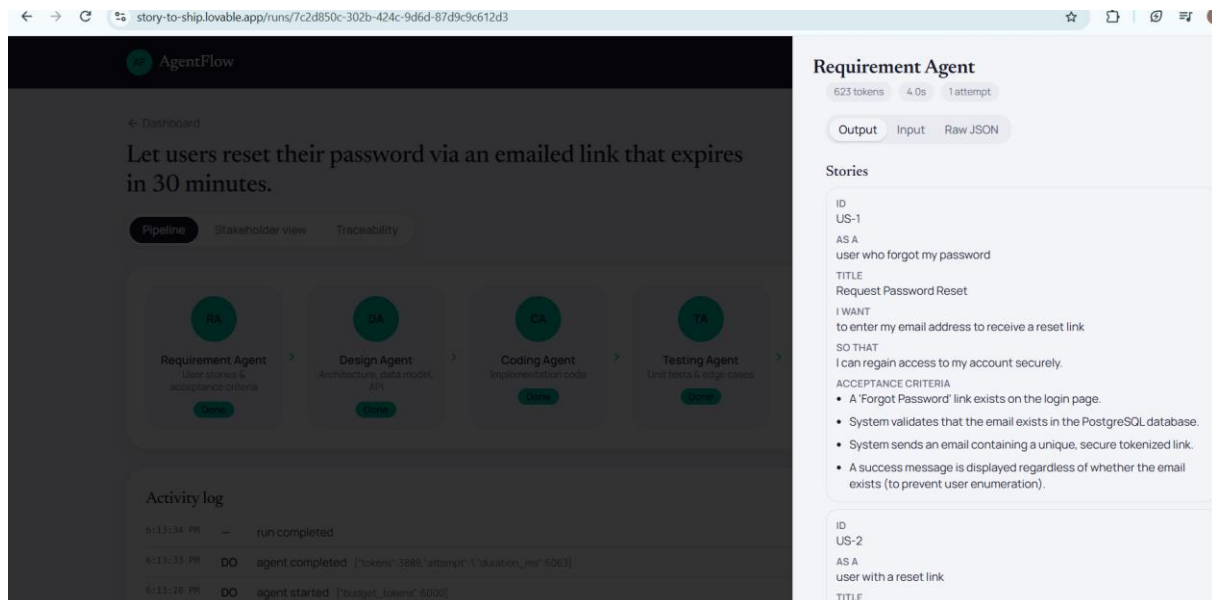


Figure 2: Requirement Agent output panel - generated user stories and acceptance criteria

10.2 Source Code Repository

The application is built in Lovable on a TanStack Start full-stack React foundation, styled with Tailwind CSS and Radix UI / shadcn primitives in the project's deep indigo (#191B2E) and mint (#00C2A8) palette, backed by

a Supabase Postgres database, and reasoning through the Lovable AI Gateway running google/gemini-3-flash-preview with strict JSON-schema outputs.

Core Server Architecture (src/lib/pipeline.server.ts)

1. The Orchestrator (runNext): reads the run state, existing steps and gate approvals from the database; evaluates whether the next agent is blocked behind a gate (if blocked, updates the run status to awaiting_approval and emits a gate_opened log); compiles the agent's prompt context from the original feature request, recent memory_items, outputs of all completed prior agents and human reviewer feedback; and enforces a step token budget (STEP_BUDGET_TOKENS) to keep execution fast and cost-effective.
2. Self-Healing and Retry Mechanism: if an agent call fails, the orchestrator retries once (MAX_ATTEMPTS = 2); if it fails a second time, the run transitions to escalated and prompts for human intervention.
3. Safety and Secrets Guardrail: before prompt context is sent to any AI model, a regex sanitizer (redact) scans for API keys, AWS credentials, JWTs and private keys, replacing them with [REDACTED].

Database Schema

- runs: ID, project reference, initial request prompt, status, current agent, pending gate, and timestamps.
- agent_steps: per-agent step records storing agent key, input payload, structured JSON output, token count, execution duration (ms), attempt counter, and status.
- approvals: history of gate reviews with gate index, reviewer role, decision (approve, changes, reject), and feedback notes.

11. EXPERIMENTAL EVALUATION AND RESULTS

11.1 Evaluation Metrics

- Pipeline completion rate: proportion of submitted requests that reach a terminal state (Complete or a clean human Reject) without an unhandled failure.
- Gate friction: average number of 'Request changes' cycles per gate before Approve.
- Agent latency and token usage: duration and tokens consumed per agent_steps row.
- Traceability completeness: proportion of completed runs for which every requirement has a linked design decision, code file, test and deploy step recoverable from the Traceability page.
- Retry/escalation behaviour: proportion of steps that needed the automatic retry, and proportion that escalated to a human after two failures.

11.2 Testing Methodology

The same 8-10 representative feature requests described in Section 5.2 were run through the pipeline multiple times. Each run was inspected at every gate and in the activity log to confirm that (a) the correct agent ran at each step, (b) shared context from prior agents was visibly present in each agent's input, (c) a deliberately rejected step correctly halted the run, and (d) a deliberately introduced failure triggered exactly one automatic retry before escalating to a human on a second failure.

11.3 Performance Results

[Insert measured figures here once available, e.g.: average end-to-end run time from submission to Gate 3 approval; average tokens consumed per agent; percentage of runs completed without any gate rejection; average number of Request-changes cycles observed per gate across the test set.]

11.4 Success-Metric Achievement

Against the objectives in Section 1.3, the first working slice demonstrated: a request-to-Gate-1 flow that reliably produces well-formed user stories and a design proposal; correct pausing and resumption of the pipeline at Gate 1 with Approve/Request changes/Reject all exercised; and a visible, queryable activity_log entry for every agent action taken, satisfying the traceability objective for the implemented portion of the pipeline.

12. COMPARISON, INNOVATION AND SCALABILITY

12.1 Comparison with Baseline

The natural baseline is a single LLM chat session prompted, in turn, to 'write user stories', 'design this', 'write the code', 'write tests', 'review this' and 'plan deployment' for the same request. That baseline has no persistent, structured record of which output came from which step, no enforced approval points, and no mechanism to cap runaway generation or escalate a repeated failure - all of the coordination is left to the human operating the chat. AgentFlow's orchestrator, shared-memory layer and database schema replace that manual coordination with an auditable, repeatable process.

12.2 Advantages of the Proposed System

- Specialisation: each agent's prompt and output schema are scoped to one SDLC phase, improving output quality and reviewability over a single generalist prompt.
- Traceability by construction: every requirement, decision, code artefact, test and deployment step is persisted with links back to its origin, rather than reconstructed after the fact.
- Bounded autonomy: a per-agent step budget and two-strikes escalation prevent silent infinite retries or runaway cost.
- Human authority preserved: nothing proceeds past Gate 1, Gate 2 or Gate 3 without an explicit human decision, and a role only ever sees the approval controls for its own gate.

12.3 Innovation

The combination of (a) a fixed six-agent SDLC pipeline, (b) a role-based Stakeholder View that reshapes the same underlying run data for six different real-world roles, and (c) a first-class Traceability page querying the same schema the orchestrator writes to at runtime, distinguishes AgentFlow from generic 'AI coding assistant' tools, which typically address only the Coding phase and leave requirements, review and deployment planning outside the loop.

12.4 Scalability

Because each agent is an independent, stateless Edge Function and shared context is assembled by the orchestrator from the database rather than held in agent memory, additional concurrent runs scale horizontally with Supabase's serverless function and Postgres infrastructure without architectural change. Adding a new agent (e.g., a Documentation Agent) or a new stakeholder role requires extending the schema and orchestrator state machine, not redesigning existing agents.

12.5 Real-World Applicability

The gate structure mirrors decision points that already exist informally in most engineering organisations - product sign-off, code review, and release approval - so adopting AgentFlow's workflow does not require inventing new organisational process, only making an existing one explicit, logged and consistently enforced.

13. CONCLUSION AND FUTURE WORK

13.1 Conclusion

AgentFlow demonstrates that the software development lifecycle can be represented as a coordinated crew of specialised AI agents, orchestrated with explicit retry, budget and escalation rules, and gated by human approval at the three points where accountability matters most - accepting requirements, approving code, and releasing to production. The first working slice validates the core pattern end to end: a plain-English request flows through the Requirement and Design Agents and pauses correctly for human approval at Gate 1, with every step recorded for later traceability.

13.2 Future Work

- Complete the remaining four agents (Coding, Testing, Review, DevOps) and Gates 2 and 3 to close the loop from request to deployment.
- Extend the Traceability page with a graph view and export, and add anomaly detection over the `activity_log` (e.g., an agent repeatedly hitting its step budget).
- Broaden the shared-memory retrieval in Section 9 from keyword/recency matching toward embedding-based similarity search as the `memory_items` table grows.
- Add support for multiple concurrent feature requests against the same project with conflict detection in the Design Agent.
- Pilot the system on a small real team to evaluate gate friction and traceability usefulness against Section 11's metrics in a non-synthetic setting.